Routing Filtered Vector Search Across CPU and Portable GPU: Completed-Call Crossovers and Eligibility Compilation

Akshat Singh Kushwaha
akshatsingh14372@outlook.com
https://github.com/a3ro-dev/qenlo

September 2026

Abstract

Filtering turns vector search into an execution-planning problem: a query must compile a predicate into an eligible set and then choose where and how to score it. We study this boundary in Qenlo, an embedded Rust/WGSL system whose FP32 rows, identifiers, metadata, tombstones, and generation remain canonical while CPU, GPU, and ANN structures are disposable derived state. On one RTX 4050 host, a controlled 100k-by-384, batch-one sweep shows a completed-call winner reversal: CPU exhaustive wins at $E = 2{,}000$ eligible rows and compact-row WGPU wins at $E = 3{,}000$. The shipped 4,096-row rule therefore selects CPU in two of six measured cells where WGPU is faster, producing descriptive P95 regret of 17.4% at $E = 3{,}000$ and 23.9% at $E = 4{,}000$. Representation also changes the winner: compact rows beat shader-side predicates at $E = 1{,}000$, whereas shader-side predicates beat compact rows at $E = 100{,}000$. We then introduce an `EligibilityPlan` that performs one canonical predicate traversal, carries execution-relevant statistics and representations, and fails closed when stale or malformed rows are presented to exact CPU search. In a separate RTX 4090/Vulkan ablation, one-pass planning reduces median complete-run P95 by 10.5–32.3% at $E \in \{1k, 4k, 6k\}$ relative to the retained two-pass path; a generation-keyed cached plan reduces it by 24.3–57.3%. These five-run sequential measurements are an implementation ablation, not a calibrated-router evaluation. One tuned USearch point is competitive with dense WGPU, but no recall–latency frontier is available. The evidence supports device- and implementation-specific routing decisions; it does not establish a universal threshold, a completed multidimensional phase map, or reduced regret from an adaptive production router.

1 Introduction

The practical input to a filtered nearest-neighbor query is not just collection size N. A metadata predicate first defines an eligible set S_P, and only $E = |S_P|$ vectors can enter the answer. Exhaustive scoring therefore performs $O(ED)$ arithmetic for dimension D. That makes a million-row collection with a one-percent filter a different execution problem from an unfiltered million-row query, even when both live in the same database.

Recent filtered-ANN work shows that selectivity, filter/query correlation, and implementation choices change the best plan [1, 4, 5, 9, 10]. This paper addresses a narrower embedded-system boundary: completed-call routing between CPU and portable-GPU exhaustive execution after filtering. Dispatch, synchronization, bounded result readback, and host predicate work are part of the measurement. The system contribution is to make eligibility a compiled, reusable plan rather than a scalar counted once for routing and traversed again for execution.

The study makes four contributions:

1. a controlled native CPU/WGPU sweep that brackets a winner reversal to $(2{,}000, 3{,}000)$ eligible vectors on one RTX 4050 host and exposes two misroutes by the shipped 4,096-row policy;

2. an `EligibilityPlan` abstraction that binds canonical generation, eligible rows, predicate shape, representation, transfer estimate, and locality statistics, with property tests and fail-closed stale-plan validation;

3. a separate RTX 4090 ablation that retains the shipped two-pass path and measures one-pass and generation-keyed cached alternatives at identical scoring, data, queries, and timing boundaries; and

4. a claim-to-artifact ledger that separates primary native evidence from Android, Intel Arc, heterogeneous library, and CUDA/FAISS context, while retaining failures and an independent FP64 oracle where claimed.

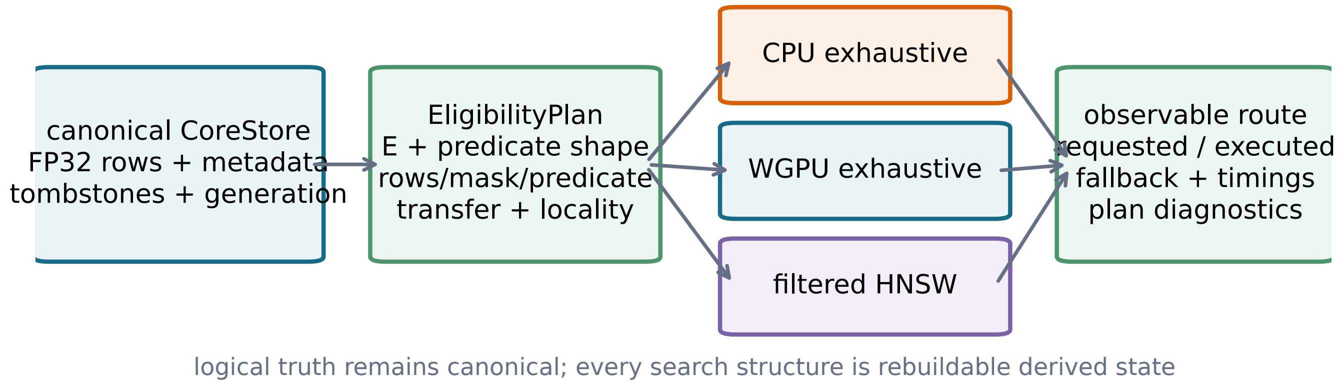

Figure 1: Canonical rows define logical truth. `EligibilityPlan` compiles one generation-bound predicate result into execution representations and diagnostics; search structures remain disposable derived state.

The central research question is: *for a fixed implementation, device, dimension, and batch policy, where does eligible work reverse the preferred exhaustive route, how does predicate representation shift that boundary, and what completed-call cost is introduced by compiling eligibility?* ANN remains secondary because the artifact contains one tuned native HNSW operating point rather than a frontier.

2 Scope and terminology

Exhaustive versus numerically exact. The CPU and GPU paths score every eligible candidate. We call them *exhaustive*. Their floating-point order can differ from an FP64 oracle near ties, so the paper reports oracle recall instead of using "exact recall" as a numerical claim. The dense native cells achieve recall@10=0.99998; the sparse native cells and qualified A6000 cells achieve 1.0.

Completed-call latency. The timed boundary begins immediately before the search call and ends after ordered identifiers and distances are available on the host. It includes predicate work inside the selected route, query transfer, dispatch, synchronization, readback, and host merge. Index build, corpus loading, and oracle construction are outside timing. Kernel-event measurements are not compared with completed calls.

The RTX 4050 crossover measures the shipped/tested revision. Its compact-row path traverses the predicate once to count eligible rows and again to materialize a fresh row-ID list. The query and row IDs are uploaded into reusable scratch buffers; vector chunks and allocations remain resident. The reported latency includes both traversals, compaction, upload, scoring, synchronization, readback, and host merge. We preserve this result as the baseline rather than overwriting it with the later eligibility-plan implementation.

Non-pooling. The native Windows, Android, Intel Arc, Linux library, real A6000, and synthetic A6000 records differ in hardware, data, implementation, or evidence quality. Every numerical comparison stays within a cohort. Cross-cohort agreement is treated as replication of a direction, not as a pooled effect.

3 System boundary

Qenlo's canonical `CoreStore` owns normalized FP32 vectors, metadata, public identifiers, tombstones, and the committed generation. Exact CPU state, filtered HNSW, WGPU buffers, and eligibility representations are derived and rebuildable (Figure 1). Route changes therefore cannot alter logical existence or recovery semantics.

The CPU path uses AVX2/FMA when available and accumulates dot products in FP64. The WGPU backend has three exhaustive predicate forms: upload compact eligible rows, upload a dense 32-bit mask, or evaluate a supported predicate in WGSL. Vectors remain device-resident. The shader scans bounded chunks, each chunk emits at most k candidates, and the host merges those lists. If an item is outside its chunk top k, at least k items in that chunk precede it under the common distance/ID order; the item cannot enter the global top k. This establishes the chunk merge but is not an algorithmic novelty claim.

Table 1: Retained evidence used by the paper. "Aggregate" means raw per-call series were not supplied.

Cohort	Timed obs.	$N \times D$	Hardware	Evidence
Windows endpoints	225,000	100k×384	RTX 4050	raw calls
Windows crossover	300,000	100k×384	RTX 4050	raw calls
Eligibility ablation	300,000	100k×384	RTX 4090	raw archive
Linux libraries	10,500	100k×384	cloud CPU	raw calls
Real strict gate	20,000	1M×768	RTX A6000	raw calls
Synthetic held-out	4,000	1M×768	RTX A6000	raw calls
Android device lab	4,144	10k/100k×384	Mali-G615	aggregate
Total	863,644			

The revised `EligibilityPlan` is compiled before execution. It records canonical generation, E, N, predicate kind, physical representation, sorted row IDs when needed, estimated transfer bytes, contiguous-run count, materialization time, traversal count, and cache state. Empty, tiny-row, sorted-row, dense-mask, and shader-predicate forms are implemented. Roaring-style and resident GPU bitsets are not. The default path traverses the canonical predicate once; the original two-pass behavior remains selectable only for measurement. A one-entry cache is keyed by exact predicate and generation. Mutations invalidate it, and exact CPU execution revalidates row ordering, liveness, and every predicate clause. A malformed or stale row list returns an error rather than producing a result.

The automatic route reports requested and executed backends, fallback, filter strategy, eligible/scored counts, transfer bytes, owned allocations, plan diagnostics, and host-side timing phases. Required-GPU mode fails rather than silently falling back. A hardware-bound `RouterProfile` can replace the static threshold only when adapter, dimension, batch size, filter mode, and cache state match; mismatches retain the 4,096-row fallback. The artifact does not yet include an automatic calibration command, persisted profiles, online adaptation, or held-out regret measurements for a calibrated profile. We therefore evaluate the shipped rule counterfactually but do not claim that the revised router reduces regret.

4 Experimental method

4.1 Evidence inventory

Table 1 summarizes the retained observations used in this revision. Counts describe artifact volume, not independent experimental replications. The Windows endpoint total includes nine completed configurations; the crossover campaign adds twelve CPU/WGPU cells. The Runpod ablation adds twelve materialization cells. An incomplete CPU development run and duplicated cloud mirrors are excluded. The real A6000 total includes a 5,000-sample cuVS run retained as a correctness-gate failure.

4.2 Primary native cohort

The Windows cohort uses 100,000 384-dimensional AG News sentence embeddings derived from a public DTU artifact [8]. Corpus, tuning, and evaluation query ranges are disjoint. Synthetic metadata is independent of vector geometry. Every retained configuration uses $k = 10$, batch one, 200 warmups, 5,000 held-out queries, and five repetitions. The endpoint cells use $E = 1{,}000$ and 100,000. The later localization sweep uses $E \in \{2k, 3k, 4k, 6k, 8k, 10k\}$ for CPU and compact-row WGPU. It was designed after inspecting the endpoints and is descriptive.

USearch expansion was selected on 1,000 tuning queries. $ef = 128$ reached tuning recall@10=0.9932 and was then frozen for the held-out range. The independent oracle scores every eligible vector in FP64 outside timing and rejects duplicate, deleted, unknown, or predicate-ineligible results.

For the primary ratios, the unit of resampling is a complete run, not an individual query. A seeded 10,000-draw bootstrap reports percentile intervals for the ratio of median run-level P95 values. Five runs produce a coarse interval; it does not model device, driver, thermal, or dataset variation.

4.3 Eligibility-materialization cohort

The matched implementation ablation ran on a separate Runpod secure-cloud host with a Ryzen 9 7950X, an RTX 4090, NVIDIA driver 570.211.01, and Vulkan. It uses the same retained 100k-by-384 dataset, independent metadata distribution, disjoint tuning and evaluation query ranges, $k = 10$, batch one, 200 warmups, 5,000 held-out calls, and

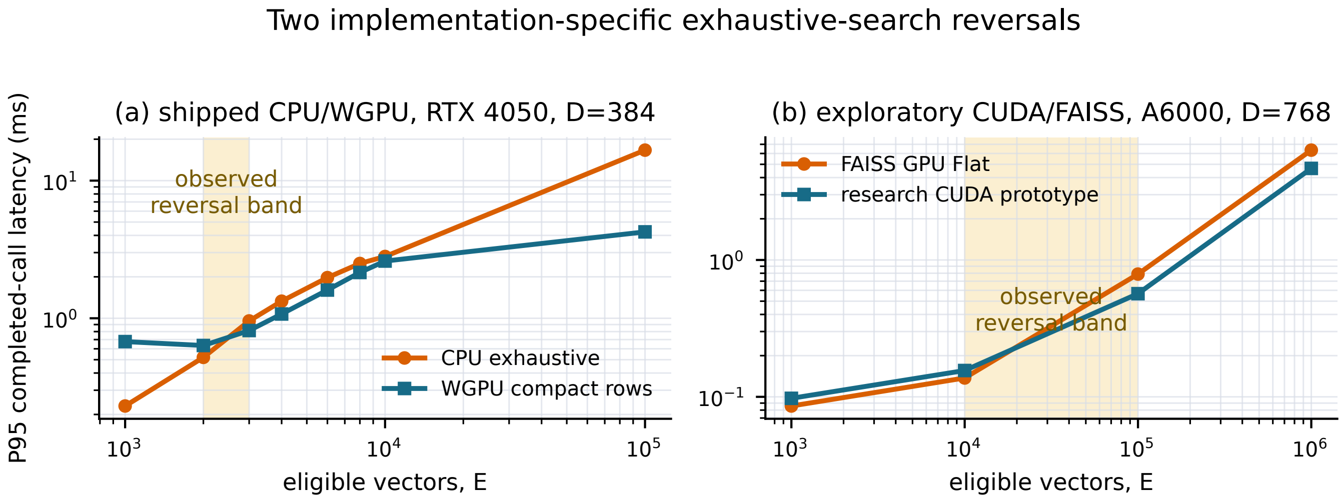

Figure 2: Two non-poolable implementation reversals. Lines connect measured points; shaded regions are brackets, not fitted universal thresholds. The left panel is shipped Rust/WGSL. The right panel is an exploratory PyTorch/CUDA prototype.

five complete runs per cell. Eligible counts are $1k, 4k, 6k, 100k$. Three modes preserve scoring and GPU execution: the original count-then-materialize path, one-pass plan compilation, and a warmed generation/predicate-keyed cached row list. Modes were executed sequentially rather than interleaved, so the comparison is descriptive and may contain time-order or thermal bias. No confidence interval is reported.

Correctness is checked against the independent FP64 oracle used by the native harness. Randomized property tests additionally compare plan output with canonical filtering across insertions, tombstones, user equality, timestamp ranges, and compound predicates. Cache reuse is tested before and after a generation-changing mutation. These tests establish semantic equivalence and stale-plan rejection; they do not establish performance generality.

4.4 External and exploratory cohorts

The Android schema-v1 records identify Android 16/API 36, a Mediatek MT6897 CPU, Mali-G615 MC6/Vulkan, and app version 0.1.0. Quick, full, and soak suites contain 16, 64, and 512 reported samples per cell. All 21 cells report recall@10=1.0, no fallback, and no failure. Power source is absent and thermal state is the uninterpreted string "0". The aggregates were transcribed without reconstructing unavailable raw distributions.

The A6000 real cohort uses one million 768-dimensional ModernBERT vectors from TopK Bench [12]. Its strict predicate yields $E = 10{,}026$. NumPy CPU, FAISS GPU Flat, and a PyTorch/CUDA research prototype each have 5,000 completed calls and an independent FP64 oracle. cuVS returned recall 0.9 and is disqualified. Predicate materialization is outside this cohort's latency boundary because every system receives the same prefiltered matrix.

The synthetic A6000 cohort uses normalized random FP32 vectors, 100 queries, 50 warmups, five repetitions, and a fresh held-out seed. It measures $E \in \{1k, 10k, 100k, 1M\}$. FAISS and the research CUDA prototype search the same matrix and have complete unordered top-10 agreement. This is cross-implementation agreement, not an independent oracle. Native WGPU produced no A6000 samples because the container exposed no usable NVIDIA Vulkan ICD.

5 Results

5.1 A native exhaustive crossover and static-policy regret

Figure 2 makes the two measured reversals literal while keeping them separate. On the RTX 4050, CPU wins at $E = 2{,}000$ (0.520 versus 0.634 ms P95), while WGPU compact rows wins at $E = 3{,}000$ (0.814 versus 0.956 ms). WGPU remains ahead through 10,000. The observed bracket is therefore $(2k, 3k)$ for this host, code revision, dimension, batch size, and predicate representation.

At $E = 100{,}000$, WGPU predicate search reaches 3.240 ms P95 versus 16.657 ms on CPU, a 5.14× ratio with whole-run bootstrap interval [3.55, 6.77]. At $E = 1{,}000$, CPU requires 0.230 ms, compact-row WGPU 0.677 ms,

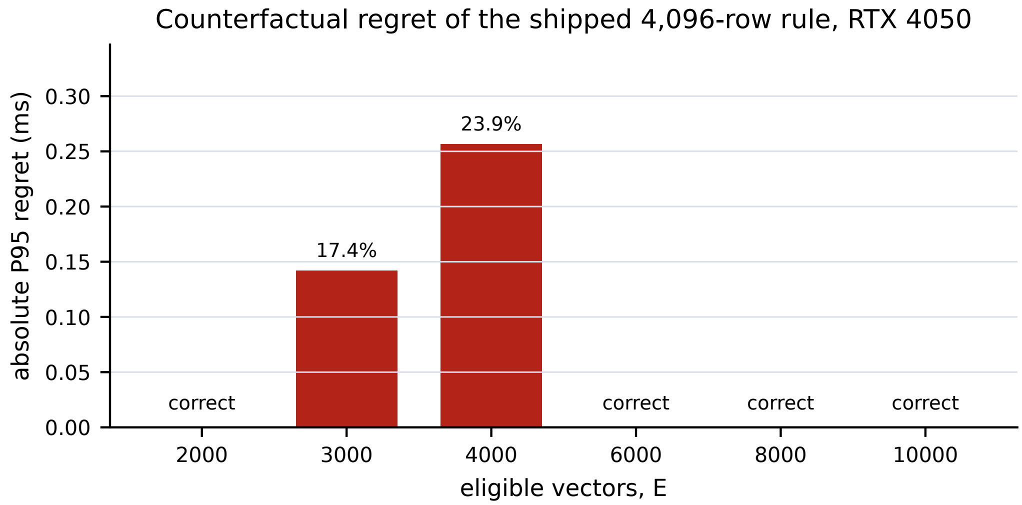

Figure 3: Descriptive P95 regret of the shipped 4,096-row rule on the six measured RTX 4050 compact-row cells. Zero bars indicate a correct route. The cells were not interleaved or collected as paired router decisions.

and shader-side predicate WGPU 2.883 ms. The endpoint reversal rules out a device-presence-only GPU routing policy. The WGPU win at $E = 3{,}000$ occurs despite the duplicated predicate traversal described in Section 2, so the measured bracket includes avoidable implementation overhead. It does not locate the crossover for a one-pass or cached-list path; that requires a matched ablation.

The shipped automatic rule chooses CPU when $E < 4{,}096$ and batch size is below eight. Applying that frozen rule to the six held-out CPU/WGPU cells gives the counterfactual in Figure 3. It chooses CPU correctly at $E = 2k$, incorrectly at $3k$ and $4k$, and WGPU correctly at $6k, 8k, 10k$. Route accuracy is therefore 4/6 cells. Relative to the fastest valid measured backend, P95 regret is 0.142 ms (17.4%) at $3k$ and 0.257 ms (23.9%) at $4k$; the unweighted six-cell means are 0.066 ms absolute and 6.9% relative. These are cell-level counterfactuals derived from separate backend runs, not paired per-query regret or evidence for a new calibrated policy.

The six localization points do not support a transferable fitted crossover. A useful execution model is

$$T_{\text{CPU}} \approx \alpha_c + \beta_c ED, \tag{1}$$
$$T_{\text{GPU}} \approx \alpha_g(B) + \beta_g ED + T_{\text{materialize}}(P, E) + T_{\text{sync}}, \tag{2}$$

but the coefficients depend on route, memory behavior, and device state. Fitting them to six post-hoc P95 points would create false precision. The measured bracket is the stronger result.

5.2 Compiling eligibility removes duplicated host work

Figure 4 preserves the original two-pass implementation as a baseline. At $E = 1k$, one-pass planning lowers median complete-run P95 from 0.131736 to 0.117870 ms (10.5%); at $4k$, from 0.305281 to 0.208009 ms (31.9%); at $6k$, from 0.401130 to 0.271618 ms (32.3%); and at $100k$, from 1.786170 to 1.720476 ms (3.7%). The warmed cache reaches 0.099747, 0.143778, 0.171330, and 1.596024 ms, respectively, corresponding to 24.3–57.3% improvement at the three sparse counts and 10.6% at the dense endpoint. Sparse cells have recall@10=1.0; the dense cells have 0.99998. No filter violation is reported.

The gain is not constant in E. Median one-pass materialization time is 12.714 microseconds at $1k$, 55.473 microseconds at $4k$, 84.528 microseconds at $6k$, and 64.851 microseconds at $100k$. The nonmonotonic dense value is retained. Predicate index behavior, row layout, allocation reuse, and synchronization contribute alongside eligible count. The cache reports zero materialization time on warmed calls, but it is valid only for an identical predicate and canonical generation. This result supports eligibility compilation and cautious caching; it does not reveal the new CPU/WGPU crossover because CPU was not part of this matched ablation.

5.3 Predicate representation is a routing variable

Figure 5 compares matched strategies. At $E = 1{,}000$, compact rows are 4.26× faster than shader-side predicate evaluation (0.677 versus 2.883 ms). At $E = 100{,}000$, the direction reverses: shader-side predicate evaluation is 1.31×

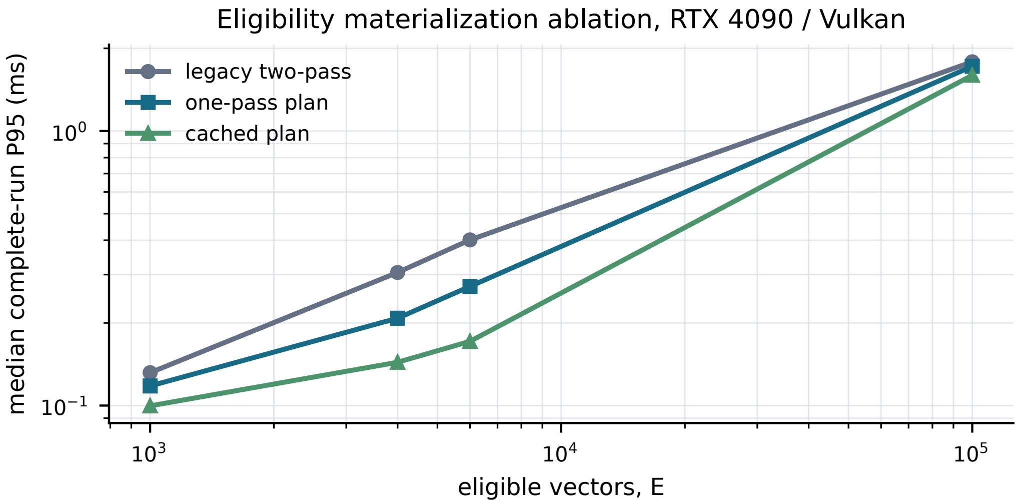

Figure 4: Matched eligibility preparation ablation on RTX 4090/Vulkan. Each point is the median of five complete-run P95 values over 25,000 held-out calls. Execution order was sequential, so no run-level uncertainty or causal generalization beyond this cohort is claimed.

faster than compact rows (3.240 versus 4.243 ms) and 1.38× faster than a mask (4.459 ms). Materializing a tiny eligible set can save device work; representing a dense set explicitly can cost more than evaluating the predicate in place. Route selection therefore has at least two decisions: choose CPU, GPU, or ANN, then, for GPU, choose rows, mask, or shader-side predicate evaluation. A router that uses only E misses this interaction.

At the dense endpoint, USearch reaches 3.625 ms P95 and recall@10=0.99224. WGPU predicate reaches 3.240 ms and 0.99998. The nominal latency ratio favors WGPU by 1.12×, but its whole-run bootstrap interval [0.77, 1.54] crosses parity. The result establishes one competitive high-recall operating point, not an ANN phase boundary. No recall–latency curve is available across E.

A separate Chroma HNSW cell on the same Windows data reaches 1.130 ms P95 and recall@10=0.99414. It uses Chroma's own storage and API rather than a route inside Qenlo, so it is not placed in Figure 5; it is retained in Appendix A. Its result matters: even at the dense endpoint where exhaustive WGPU is competitive with USearch, a different HNSW implementation is substantially faster. The evidence does not support "exhaustive beats ANN" as a system-independent statement.

5.4 Android and non-NVIDIA evidence

Figure 6 replaces the previous heterogeneous seven-bar chart with matched panels. CPU wins the 10k-row quick cell at P95 (5.767 versus 12.487 ms). WGPU wins the 100k full cell (16.975 versus 29.436 ms) and the 512-sample soak (17.678 versus 27.019 ms). Because both row count and suite length change, this is a qualitative reversal, not a mobile crossover bracket.

In the matched 100k soak, exhaustive WGPU also has lower P95 than IVF-Flat (21.453 ms) and IVF-SQ8 (37.090 ms). This does not prove that exhaustive search dominates IVF: only one parameterization and one generated workload were observed. Dense WGPU batch eight reports 11.131 ms versus 17.678 ms at batch one, but the emitted batch statistic is not normalized per query. It supports amortization as a route feature, not a per-query speedup claim.

An independent Intel Arc soak at 100k×384 reports 4.444 ms P95 for exhaustive WGPU and 16.486 ms for CPU, both at recall@10=1.0 over 512 samples. This is a second non-NVIDIA execution result. It is not enough to estimate a device-family effect.

5.5 The exploratory A6000 reversal

At the real TopK point $E = 10{,}026$, FAISS GPU Flat leads the qualified systems: 0.132 ms P95 versus 0.171 ms for the research CUDA exhaustive prototype and 0.197 ms for NumPy CPU. All three reach oracle recall@10=1.0 over 5,000 calls. The prototype is 1.29× slower than FAISS. Every manuscript occurrence now names it "research CUDA exhaustive prototype" because it is PyTorch code, not the shipped Qenlo backend.

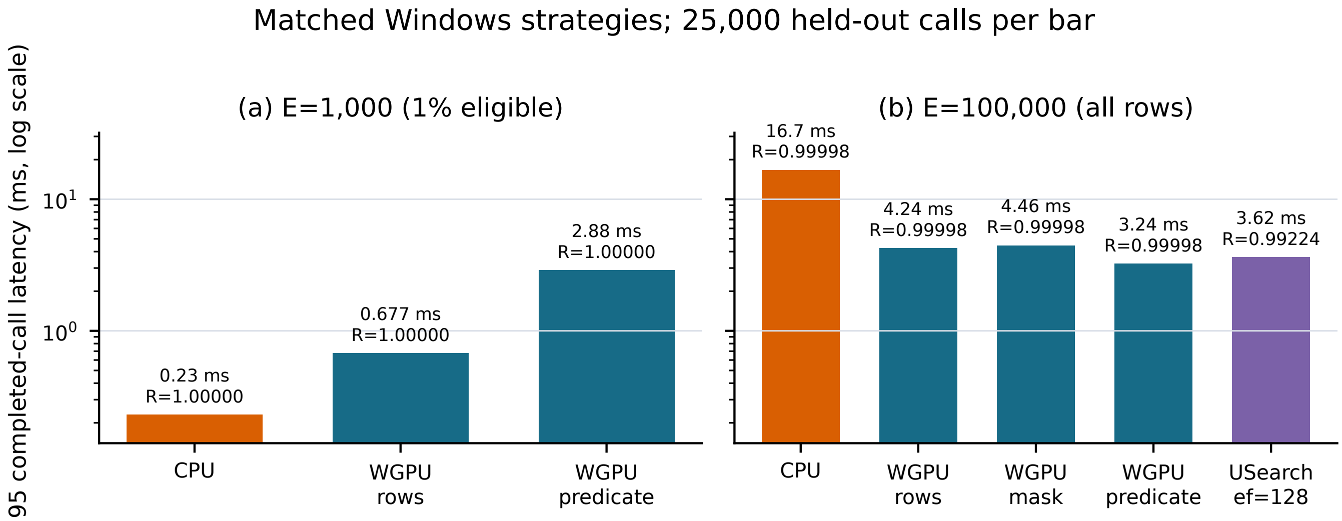

Figure 5: Matched native Windows strategies. All exhaustive results score the same eligible set; the USearch point is approximate. Recall is measured against the same FP64 oracle.

Table 2: Hypothesis status after the retained campaigns.

	Hypothesis	Status
H1	Exhaustive GPU can beat CPU and compete with high-recall HNSW	Partial: native cells support it; the single HNSW interval crosses parity
H2	E predicts exhaustive work better than N	Open: E varies, but matched E across N does not
H3	Fixed overhead creates a CPU/GPU reversal	Supported on RTX 4050; bracket is device-specific
H4	Batching shifts the reversal	Descriptive Android/Arc support; no matched native batch matrix
H5	High-recall ANN can lose its latency advantage	One controlled point only; no Pareto curve
H6	Adaptive routing reduces regret	Open: the static rule misroutes 2/6 cells; no frozen calibrated policy was evaluated

The fresh-seed synthetic sweep produces a different result at larger E. FAISS is faster at 1k and 10k; the buffered prototype is faster at 100k (0.567 versus 0.789 ms) and 1M (4.668 versus 6.347 ms). The observed reversal lies in $(10k, 100k)$. The relative gain is stable at the two largest points (1.39× and 1.36×), but no intermediate cell localizes the boundary further. This is evidence about two GPU formulations, not a general PyTorch-versus-FAISS result.

5.6 What the broader archive adds

The descriptive Linux cohort holds data and host fixed for seven libraries, 1,500 calls each. FAISS HNSW is fastest at 1.494 ms P95 and recall 0.9994. Chroma HNSW reaches 3.777 ms/0.9984, USearch 5.093 ms/0.9914, FAISS Flat 10.217 ms/1.0, Qenlo CPU 12.901 ms/1.0, Milvus Lite AUTOINDEX 76.229 ms/0.9988, and LanceDB Flat 242.256 ms/1.0. Figure 7, moved to Appendix A, is context because build, persistence, and API semantics differ. The cohort shows that implementation boundaries can dominate the label "exhaustive" or "ANN." It is not a product leaderboard.

6 Hypotheses and inference limits

The experiments were initially organized around six hypotheses. Table 2 states their final status without converting absent cells into positive evidence.

Three inferences survive this audit. First, eligible count is useful but insufficient: representation, materialization, D, B, and device state alter completed-call cost. The fixed-E, varying-N experiment needed to rank E against N remains absent. Second, a threshold must be calibrated to the concrete implementation. The native reversal occurs

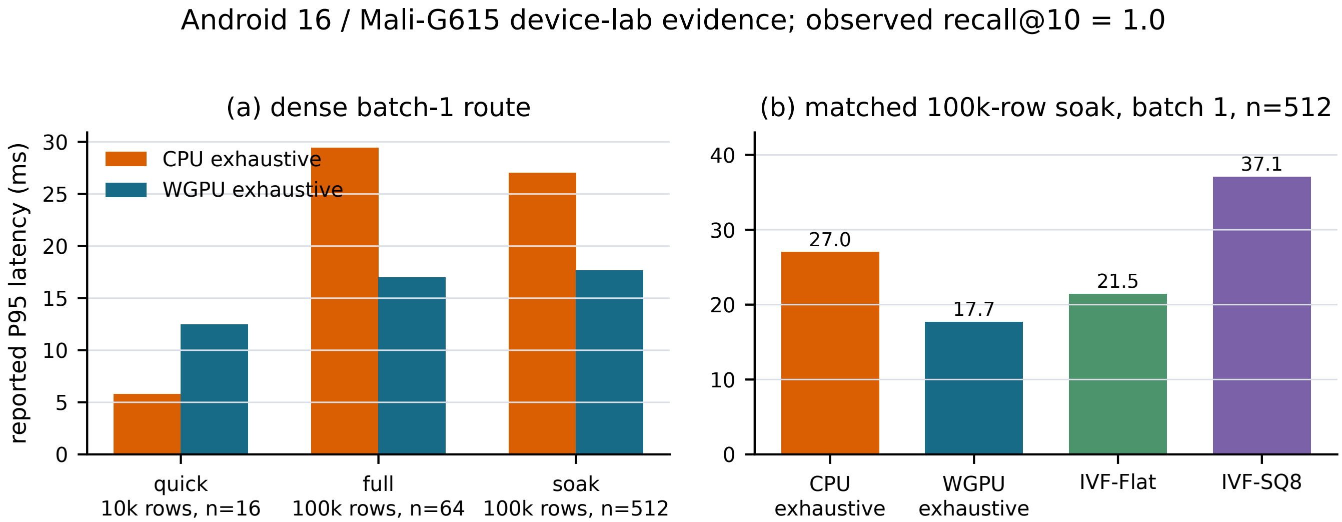

Figure 6: Android device-lab aggregates. Panel (a) reports dense batch-one CPU/WGPU cells. Panel (b) holds corpus, suite, and batch fixed. The IVF rows happened to return recall@10=1.0 on this workload; they remain approximate algorithms.

around a few thousand 384-dimensional vectors, while the unrelated A6000 formulation reverses one to two orders of magnitude later at $D = 768$. Third, an ANN route should be selected from a recall–latency frontier, not one tuned point. The current artifact cannot determine when filtered ANN becomes Pareto-optimal as E changes.

7 Related work and novelty boundary

FAISS provides optimized exhaustive and approximate CPU/GPU search [3, 6]. HNSW exposes a recall/latency tradeoff through graph expansion [7]. DiskANN and SPANN target much larger, storage-aware regimes [2, 11]. Filtered-DiskANN and ACORN modify graph construction or traversal to preserve access under predicates [4, 9].

Recent benchmarks make broad novelty claims about “exact sometimes wins” untenable. Iff et al. benchmark eleven FANNS methods over transformer embeddings, filter types, scales, and selectivities and report that method choice is workload-dependent [5]. Shi et al. emphasize unified tuning, query difficulty, selectivity, and dataset dependence [10]. Amanbayev et al. report hybrid approximate/exhaustive execution and pgvector cases where an optimizer selects ANN despite a comparable exhaustive plan with perfect recall [1].

Against that literature, this paper’s contribution is not the observation that selective filters can favor a scan. It is the measured CPU-to-portable-GPU exhaustive boundary after filtering, the completed-call cost of predicate representation and eligibility compilation, and the resulting static-policy regret on held-out cells. The present evidence is less complete than a FANNS benchmark: it lacks filter-aware ANN methods, recall curves across selectivity, correlation regimes, and scale. It is more specific about one embedded execution boundary, canonical-state safety, fallback observability, and artifact traceability.

8 Threats to validity

The primary crossover uses one laptop, one dimension, one batch size, one corpus, and metadata independent of vector geometry. The six-point sweep is post-hoc and backend order was not interleaved. The shipped compact-row baseline performs two predicate traversals per call. The separate RTX 4090 ablation shows that one-pass and cached preparation change latency, but it omits CPU and cannot relocate the RTX 4050 crossover. The CPU implementation uses AVX2/FMA with FP64 accumulation; it is a valid Qenlo product baseline, not a state-of-the-art CPU boundary. FAISS CPU Flat, optimized FP32 SIMD, and a controlled parallel exact baseline are absent from the matched Windows sweep.

The Android evidence is author-supplied aggregate output without raw series, controlled thermal state, or power telemetry. The Intel Arc record is one device-lab soak. The A6000 prototype is unshipped and excludes predicate materialization in the real TopK cell. Process isolation between FAISS and PyTorch remains a threat despite matched data, query order, hardware, and timing boundary.

The study has one controlled HNSW operating point and no genuinely filter-aware ANN system in the native selectivity sweep. Metadata correlation, N at fixed E, $D = 768$ on the primary host, concurrency, energy, memory peaks, and a frozen calibrated-router evaluation remain unmeasured. The static-regret calculation compares per-cell P95 from separate runs and gives six cells equal weight; it is not a production workload distribution. The correct next experiment is a same-host, randomized sweep over E, D, batch size, and recall that compares optimized CPU exhaustive, WGPU representations, FAISS Flat, HNSW curves, and at least one filter-aware ANN method. Until then, the evidence is a crossover study with an eligibility-planning ablation, not a multidimensional phase map.

9 Conclusion

On one RTX 4050 and one 100k-by-384 workload, shipped CPU and compact-row WGPU exhaustive search exchange the lead between 2,000 and 3,000 eligible vectors. The shipped 4,096-row policy consequently misroutes two of six measured cells, with up to 23.9% descriptive P95 regret. Representation is part of the decision: compact rows win over shader-side evaluation when the eligible set is tiny, then lose when it is dense. A separate RTX 4090 ablation shows that compiling eligibility once removes measurable completed-call cost while preserving the original path as a baseline; generation-keyed caching helps further when reuse is valid.

This revision establishes a safer substrate for adaptive execution and quantifies one static-policy failure. It does not establish that a calibrated router reduces held-out regret. At the dense endpoint, exhaustive WGPU is competitive with one tuned USearch point, but the interval crosses parity and no ANN frontier exists. The defensible result is therefore a bounded systems finding: filtered vector search should compile eligibility and route with device- and representation-specific evidence, not a universal row threshold.

Reproducibility statement

For the primary native, Linux, and A6000 cohorts, the repository retains raw per-call samples alongside run summaries, configurations, tuning choices, oracle outputs, checksums, environment captures, failed backends, processed tables, plotting code, and reproduction commands. Android evidence is retained as the supplied aggregate records. Generated figures are derived from CSV/JSON artifacts by `research/scripts/analyze_results.py` and `generate_plots.py`. Appendix B maps every headline claim to its artifact family.

References

[1] Abylay Amanbayev, Brian Tsan, Tri Dang, and Florin Rusu. Filtered approximate nearest neighbor search in vector databases: System design and performance analysis. *arXiv preprint arXiv:2602.11443*, 2026. URL https://arxiv.org/abs/2602.11443.

[2] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. SPANN: Highly-efficient billion-scale approximate nearest neighborhood search. In *Advances in Neural Information Processing Systems*, volume 34, pages 5199–5212, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/299dc35e747eb77177d9cea10a802da2-Abstract.html.

[3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *arXiv preprint arXiv:2401.08281*, 2024. URL https://arxiv.org/abs/2401.08281.

[4] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, and Yin Zhang. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*, pages 3406–3416, 2023. doi: 10.1145/3543507.3583552.

[5] Patrick Iff, Paul Brügger, Marcin Chrapek, David Kochergin, Maciej Besta, and Torsten Hoefler. Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 610–622, 2026. doi: 10.1145/3805712.3809731. URL https://doi.org/10.1145/3805712.3809731.

[6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.

[7] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.

[8] Beatrix Miranda Ginn Nielsen. DTU pretrained sentence BERT AG News embeddings, 2022.

[9] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data*, 2(3), 2024. doi: 10.1145/3654923.

[10] Jiayang Shi, Yuzheng Cai, and Weiguo Zheng. Filtered approximate nearest neighbor search: A unified benchmark and systematic experimental study. *arXiv preprint arXiv:2509.07789*, 2025. URL https://arxiv.org/abs/2509.07789.

[11] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html.

[12] TopK. TopK Bench: Vector database benchmarks. https://github.com/topk-io/bench, 2026. Commit 398ce7d72dea7ad765ada54c5daa014c498efb52; accessed 2026-09-02.

A Complete retained result tables

A.1 Native Windows endpoint and crossover cells

Table 3: Windows 100k×384 endpoint context. Each row contains five runs and 25,000 timed held-out calls; Chroma uses its own storage/API boundary.

Eligible	System	P95 ms	Recall@10	Search
1,000	CPU exhaustive	0.2304	1.00000	exhaustive
1,000	WGPU compact rows	0.6766	1.00000	exhaustive
1,000	WGPU predicate	2.8832	1.00000	exhaustive
100,000	CPU exhaustive	16.6567	0.99998	exhaustive
100,000	WGPU compact rows	4.2428	0.99998	exhaustive
100,000	WGPU mask	4.4591	0.99998	exhaustive
100,000	WGPU predicate	3.2404	0.99998	exhaustive
100,000	USearch, ef=128	3.6245	0.99224	ANN
100,000	Chroma, ef=128	1.1303	0.99414	ANN

Table 4: Post-hoc native crossover sweep. Each row contains five runs and 25,000 held-out calls per backend.

Eligible	CPU P95 ms	WGPU-row P95 ms	CPU/WGPU	Recall@10
2,000	0.5198	0.6340	0.8199	1.0
3,000	0.9563	0.8144	1.1742	1.0
4,000	1.3305	1.0737	1.2392	1.0
6,000	1.9735	1.6033	1.2309	1.0
8,000	2.5003	2.1508	1.1625	1.0
10,000	2.8104	2.6041	1.0792	1.0

A.2 Static-policy counterfactual

Table 5: Shipped 4,096-row rule applied to the six RTX 4050 crossover cells. Regret is relative to the fastest valid measured backend at cell-level P95.

Eligible	Chosen	Fastest	Absolute regret ms	Relative regret
2,000	CPU	CPU	0.0000	0.0%
3,000	CPU	WGPU rows	0.1419	17.4%
4,000	CPU	WGPU rows	0.2568	23.9%
6,000	WGPU rows	WGPU rows	0.0000	0.0%
8,000	WGPU rows	WGPU rows	0.0000	0.0%
10,000	WGPU rows	WGPU rows	0.0000	0.0%

A.3 Runpod eligibility-materialization ablation

Table 6: RTX 4090/Vulkan ablation. Each row contains five complete runs and 25,000 held-out calls. P50/P95/P99 are medians of run-level percentiles.

Eligible	Preparation	P50 ms	P95 ms	P99 ms	Recall@10
1,000	legacy two-pass	0.120365	0.131736	0.158396	1.00000
1,000	one-pass	0.109695	0.117870	0.145292	1.00000
1,000	cached	0.090789	0.099747	0.113172	1.00000
4,000	legacy two-pass	0.290994	0.305281	0.325118	1.00000
4,000	one-pass	0.198161	0.208009	0.224970	1.00000
4,000	cached	0.134842	0.143778	0.158156	1.00000
6,000	legacy two-pass	0.385110	0.401130	0.440043	1.00000
6,000	one-pass	0.256079	0.271618	0.290302	1.00000
6,000	cached	0.162404	0.171330	0.194143	1.00000
100,000	legacy two-pass	1.663349	1.786170	1.903128	0.99998
100,000	one-pass	1.607435	1.720476	1.816527	0.99998
100,000	cached	1.537885	1.596024	1.690691	0.99998

A.4 Android device-lab aggregates

Table 7: All 21 Android cells. B is batch size; n is the reported sample count. Every cell reports recall@10=1.0 and no fallback.

Suite	Cell	Rows	B	n	P50	P95	P99 ms
quick	CPU exhaustive	10k	1	16	1.841	5.767	5.767
quick	WGPU exhaustive	10k	1	16	8.707	12.487	12.487
quick	WGPU exhaustive	10k	8	16	5.886	5.886	5.886
quick	IVF-Flat	10k	1	16	3.616	8.056	8.056
quick	IVF-SQ8	10k	1	16	4.746	7.316	7.316
quick	auto selective CPU	10k	1	16	0.023	0.025	0.025
quick	auto selective WGPU	10k	8	16	2.310	2.310	2.310
full	CPU exhaustive	100k	1	64	18.308	29.436	41.559
full	WGPU exhaustive	100k	1	64	13.806	16.975	48.375
full	WGPU exhaustive	100k	8	64	10.608	11.101	11.101
full	IVF-Flat	100k	1	64	8.851	21.617	32.431
full	IVF-SQ8	100k	1	64	20.394	26.578	29.126
full	auto selective CPU	100k	1	64	0.256	0.311	0.323
full	auto selective WGPU	100k	8	64	10.039	15.571	15.571
soak	CPU exhaustive	100k	1	512	25.799	27.019	27.572
soak	WGPU exhaustive	100k	1	512	14.274	17.678	25.769
soak	WGPU exhaustive	100k	8	512	10.290	11.131	12.388
soak	IVF-Flat	100k	1	512	13.977	21.453	24.549
soak	IVF-SQ8	100k	1	512	27.359	37.090	40.581
soak	auto selective CPU	100k	1	512	0.357	0.544	0.743
soak	auto selective WGPU	100k	8	512	8.174	9.890	12.441

The automatic rows use a selective predicate and cannot be compared with dense rows. Their fixed-policy counterfactuals were not retained, so routing regret is not computable. Upload/readback/allocation diagnostics remain in `android_device_lab.csv`.

A.5 A6000 result distributions

Table 8: Real TopK strict-filter cell, $N = 1M$, $D = 768$, $E = 10{,}026$, 5,000 calls/system.

System	P50	P95	P99 ms	Recall@10	Gate
NumPy CPU exhaustive	0.1217	0.1973	0.2283	1.0	pass
FAISS GPU Flat	0.1261	0.1322	0.1458	1.0	pass
research CUDA exhaustive prototype	0.1600	0.1707	0.1809	1.0	pass
cuVS brute force	0.3615	0.3846	0.4131	0.9	fail

Table 9: Held-out synthetic A6000 completed-call distribution, 500 calls/system/cell. QPS is derived from mean latency.

Eligible	System	P50	P95	P99 ms	QPS
1k	FAISS Flat	0.0672	0.0857	0.0952	14,446
1k	research CUDA prototype	0.0775	0.0975	0.1155	12,303
10k	FAISS Flat	0.1272	0.1373	0.1505	7,854
10k	research CUDA prototype	0.1438	0.1557	0.1741	6,911
100k	FAISS Flat	0.7755	0.7887	0.8001	1,286
100k	research CUDA prototype	0.5559	0.5673	0.5763	1,797
1M	FAISS Flat	6.2736	6.3475	6.4357	159
1M	research CUDA prototype	4.6508	4.6683	4.7364	214

A.6 Descriptive Linux library cohort

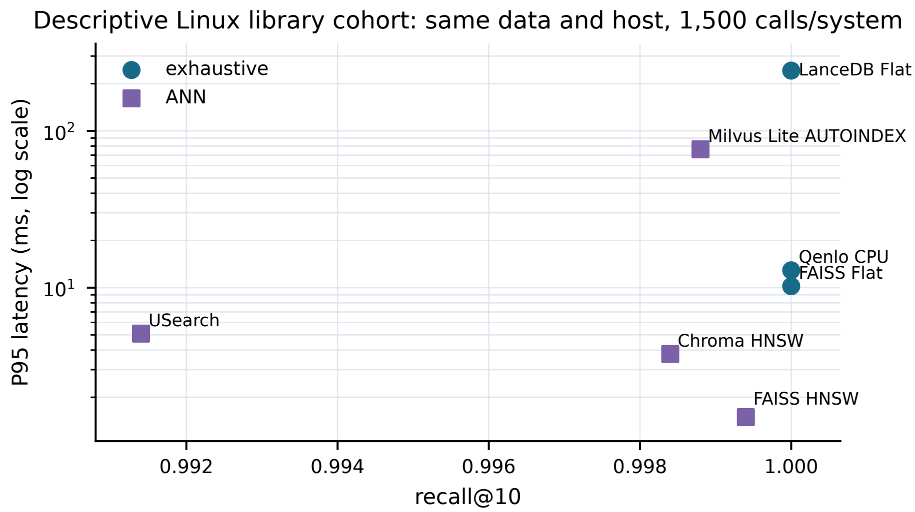

Figure 7: Descriptive Linux cohort. Latency and recall are comparable within this host, but service and persistence semantics differ.

Table 10: All-row Linux 100k×384 cohort, three runs and 1,500 calls/system.

System	Search	P95 ms	Recall@10
FAISS HNSW	ANN	1.4939	0.9994
Chroma HNSW	ANN	3.7767	0.9984
USearch	ANN	5.0933	0.9914
FAISS Flat	exhaustive	10.2171	1.0000
Qenlo CPU	exhaustive	12.9014	1.0000
Milvus Lite AUTOINDEX	ANN	76.2294	0.9988
LanceDB Flat	exhaustive	242.2557	1.0000

B Artifact map

Table 11: Claim-to-artifact map. Paths are relative to the repository root.

Claim family	Authoritative artifact
Native Windows endpoints	`benchmarks/2026-08-28/real/*/samples.csv`, `runs.csv`, `summary.*`
Native 2k–10k crossover	`research/data/raw/2026-09-02-native-crossover/`
Static-router regret	`research/data/processed/static_router_regret.csv`, regenerated from the native crossover summaries
Eligibility-plan ablation	`research/artifacts/qenlo-phase1-eligibility-2026-09-04.tar.gz`; processed CSV under `research/data/processed/`
Android aggregates	Processed CSV under `research/data/processed/`; raw provenance note under `research/data/raw/2026-09-02-android/`
Linux libraries	`benchmark-results/2026-09-02-runpod-archive/retained-archive/artifacts/`
Real A6000 strict gate	`benchmark-results/2026-09-02-runpod-a6000-strict-research-gate/`
Synthetic A6000 sweep	`research/data/raw/2026-09-02-a6000-cuda/heldout/`
Processed tables	`research/data/processed/`
Analysis and figures	`research/scripts/analyze_results.py`, `generate_plots.py`
Failures and exclusions	`research/methodology-audit.md`, `completion-audit.md`

C Minimum next experiment

The missing experiment is a same-machine phase diagram with fixed data and query order. It should test eligible counts of 500, 1k, 2k, 3k, 5k, 10k, 25k, 50k, and 100k; dimensions 384 and 768; and batches 1, 8, and 32. Every cell should compare optimized CPU exhaustive, shipped WGPU exhaustive, FAISS GPU Flat, a tuned HNSW recall–latency curve, and at least one filter-aware ANN method. Headline cells should use at least 20 independent complete runs with interleaved backend order. The retained eligibility ablation should be extended with a resident GPU representation and matched CPU cells. A calibrated profile must be frozen on tuning queries, then evaluated as actual route decisions against always-CPU, always-GPU, the 4,096-row rule, and an evaluation-only oracle. Repeating identical E at different N would test H2; correlated and anti-correlated metadata would test whether selectivity alone predicts ANN difficulty.